PRÁTICA LABORATORIAL 06

Objetivos:
- Listas Ligadas

EXERCÍCIOS – GRUPO I

Estrutura básica

```
NÓ {
   dados: INTEIRO
   proximo: PONTEIRO para NÓ
}

// Variável global
inicio: PONTEIRO para NÓ ← NULL
```

Exercício 1: Criar uma lista ligada para armazenar números inteiros. Implementar um procedimento que insira 5 números no início da lista e depois mostre todos os elementos. Verificar que os números aparecem em ordem inversa à inserção.

```
PROCEDIMENTO inserir_inicio(valor: INTEIRO)
INÍCIO
   novo_no: NÓ
   novo_no.dados ← valor
   novo_no.proximo ← inicio
   inicio ← novo_no
   ESCREVER "Inserido", valor, "no início"
FIM

PROCEDIMENTO mostrar_lista()
INÍCIO
   SE (inicio = NULL) ENTÃO
      ESCREVER "Lista vazia"
      RETORNAR
   FIM SE

   atual ← inicio
   ESCREVER "Lista: "
   ENQUANTO (atual ≠ NULL) FAÇA
      ESCREVER atual.dados, " -> "
      atual ← atual.proximo
   FIM ENQUANTO
   ESCREVER "NULL"
FIM

// Programa principal
INÍCIO
   ESCREVER "=== EXERCÍCIO 1 - INSERÇÃO NO INÍCIO ==="
```

```
   // Inserir 5 números
   inserir_inicio(10)
   inserir_inicio(20)
   inserir_inicio(30)
   inserir_inicio(40)
   inserir_inicio(50)

   ESCREVER "Ordem de inserção: 10, 20, 30, 40, 50"
   mostrar_lista()
   ESCREVER "Ordem na lista: 50, 40, 30, 20, 10 (inversa!)"
FIM
```

Exercício 2: Desenvolver um sistema de lista de tarefas usando lista ligada. Criar procedimentos para: adicionar tarefa no fim da lista, remover uma tarefa específica, e mostrar todas as tarefas pendentes. Testar com pelo menos 4 tarefas.

```
PROCEDIMENTO inserir_fim(tarefa: TEXTO)
INÍCIO
   novo_no: NÓ
   novo_no.dados ← tarefa
   novo_no.proximo ← NULL

   SE (inicio = NULL) ENTÃO
      inicio ← novo_no
   SENÃO
      atual ← inicio
      ENQUANTO (atual.proximo ≠ NULL) FAÇA
         atual ← atual.proximo
      FIM ENQUANTO
      atual.proximo ← novo_no
   FIM SE

   ESCREVER "Tarefa '", tarefa, "' adicionada"
FIM

PROCEDIMENTO remover_tarefa(tarefa: TEXTO)
INÍCIO
   SE (inicio = NULL) ENTÃO
      ESCREVER "Lista de tarefas vazia"
      RETORNAR
   FIM SE

   // Remover do início
   SE (inicio.dados = tarefa) ENTÃO
      temp ← inicio
      inicio ← inicio.proximo
      ESCREVER "Tarefa '", tarefa, "' removida"
      RETORNAR
   FIM SE

   // Procurar no resto da lista
   atual ← inicio
   ENQUANTO (atual.proximo ≠ NULL E atual.proximo.dados ≠ tarefa) FAÇA
      atual ← atual.proximo
   FIM ENQUANTO

   SE (atual.proximo ≠ NULL) ENTÃO
```

```
      temp ← atual.proximo
      atual.proximo ← temp.proximo
      ESCREVER "Tarefa '", tarefa, "' removida"
   SENÃO
      ESCREVER "Tarefa '", tarefa, "' não encontrada"
   FIM SE
FIM

PROCEDIMENTO mostrar_tarefas()
INÍCIO
   SE (inicio = NULL) ENTÃO
      ESCREVER "Nenhuma tarefa pendente"
      RETORNAR
   FIM SE

   atual ← inicio
   contador ← 1
   ESCREVER "=== TAREFAS PENDENTES ==="
   ENQUANTO (atual ≠ NULL) FAÇA
      ESCREVER contador, ".", atual.dados
      atual ← atual.proximo
      contador ← contador + 1
   FIM ENQUANTO
FIM

// Programa principal
INÍCIO
   ESCREVER "=== EXERCÍCIO 2 - LISTA DE TAREFAS ==="

   // Adicionar tarefas
   inserir_fim("Estudar para teste")
   inserir_fim("Fazer compras")
   inserir_fim("Ligar ao médico")
   inserir_fim("Pagar contas")

   mostrar_tarefas()

   // Remover uma tarefa
   remover_tarefa("Fazer compras")

   ESCREVER ""
   mostrar_tarefas()
FIM
```

Exercício 3: Implementar uma função que conte quantos elementos existem numa lista ligada e outra função que procure um número específico na lista. Se encontrar, mostrar em que posição está (começando em 0). Se não encontrar, mostrar mensagem apropriada.

```
FUNÇÃO contar_elementos(): INTEIRO
INÍCIO
   contador ← 0
   atual ← inicio

   ENQUANTO (atual ≠ NULL) FAÇA
      contador ← contador + 1
      atual ← atual.proximo
   FIM ENQUANTO

   RETORNAR contador
FIM

FUNÇÃO procurar_elemento(valor: INTEIRO): INTEIRO
INÍCIO
   atual ← inicio
   posicao ← 0

   ENQUANTO (atual ≠ NULL) FAÇA
      SE (atual.dados = valor) ENTÃO
         RETORNAR posicao
      FIM SE
      atual ← atual.proximo
      posicao ← posicao + 1
   FIM ENQUANTO

   RETORNAR -1  // Não encontrado
FIM

PROCEDIMENTO testar_busca(valor: INTEIRO)
INÍCIO
   posicao ← procurar_elemento(valor)

   SE (posicao = -1) ENTÃO
      ESCREVER "Número", valor, "não encontrado na lista"
   SENÃO
      ESCREVER "Número", valor, "encontrado na posição", posicao
   FIM SE
FIM

// Programa principal
INÍCIO
   ESCREVER "=== EXERCÍCIO 3 - CONTAR E PROCURAR ==="

   // Criar lista de teste
   inserir_fim(15)
   inserir_fim(30)
   inserir_fim(45)
   inserir_fim(60)
   inserir_fim(75)

   mostrar_lista()
```

```
   // Contar elementos
   total ← contar_elementos()
   ESCREVER "Total de elementos:", total

   // Procurar elementos
   testar_busca(45)  // Deve encontrar
   testar_busca(99)  // Não deve encontrar
   testar_busca(15)  // Deve encontrar na posição 0
   testar_busca(75)  // Deve encontrar na última posição
FIM
```

Exercício 4: Criar um procedimento que remova todos os números pares de uma lista ligada, mantendo apenas os números ímpares. Mostrar a lista antes e depois da remoção para verificar o resultado.

```
PROCEDIMENTO remover_pares()
INÍCIO
   SE (inicio = NULL) ENTÃO
      ESCREVER "Lista vazia"
      RETORNAR
   FIM SE

   // Remover pares do início
   ENQUANTO (inicio ≠ NULL E inicio.dados % 2 = 0) FAÇA
      temp ← inicio
      inicio ← inicio.proximo
      ESCREVER "Removido par:", temp.dados
   FIM ENQUANTO

   // Remover pares do resto da lista
   SE (inicio ≠ NULL) ENTÃO
      atual ← inicio
      ENQUANTO (atual.proximo ≠ NULL) FAÇA
         SE (atual.proximo.dados % 2 = 0) ENTÃO
            temp ← atual.proximo
            atual.proximo ← temp.proximo
            ESCREVER "Removido par:", temp.dados
         SENÃO
            atual ← atual.proximo
         FIM SE
      FIM ENQUANTO
   FIM SE
FIM

// Programa principal
INÍCIO
   ESCREVER "=== EXERCÍCIO 4 - REMOVER PARES ==="

   // Criar lista com números pares e ímpares
   inserir_fim(12)  // par
   inserir_fim(7)   // ímpar
   inserir_fim(20)  // par
   inserir_fim(15)  // ímpar
   inserir_fim(8)   // par
   inserir_fim(33)  // ímpar
```

```
   inserir_fim(14)  // par

   ESCREVER "Lista original:"
   mostrar_lista()

   // Remover todos os pares
   remover_pares()

   ESCREVER "Lista após remover pares:"
   mostrar_lista()
   ESCREVER "Restaram apenas números ímpares: 7, 15, 33"
FIM
```

Bom trabalho! ☺